

Executable Unpacker: A Debugger-Based Runtime Memory Dump Tool for Packed Windows PE Executables

Student Name Surname

Department of Computer Engineering
University Name — Ankara, Turkey

Abstract—This paper presents a tool designed to unpack packed Windows Portable Executable (PE) files by dumping their runtime memory image and reconstructing a valid executable on disk. The tool consists of four main components: PEParse (static analysis), DebuggerEngine (dynamic OEP detection), Dumper (memory section capture), and IATRebuilder (import table reconstruction). Implemented in C++20 adhering to SOLID design principles, the tool supports both a command-line interface (CLI) and a native Win32 GUI. The Original Entry Point

(OEP) is detected dynamically via hardware breakpoints (DR0–DR3) and the PAGE_GUARD mechanism; memory sections are then written to disk as a runnable dump.

Keywords—PE file, unpacker, debugging, OEP detection, IAT reconstruction, Windows security, C++20, object-oriented programming

I. INTRODUCTION

Software packers compress or encrypt PE executables to hinder reverse-engineering of malware and commercial applications alike. A packed executable decompresses itself into memory at runtime and jumps to the original code (OEP — Original Entry Point), presenting a significant barrier to static analyzers.

The **Executable Unpacker** developed in this project leverages the Windows Debugging API to execute the target process, capture the OEP moment, and write the in-memory image as a valid PE file to disk. The tool is designed for defensive purposes such as security research, malware analysis, and reverse-engineering education.

The remainder of this report is organized as follows: Section II covers system architecture and design decisions; Section III details each component; Section IV describes the GUI implementation; Section V covers build and usage; and Section VI presents conclusions.

II. SYSTEM ARCHITECTURE AND DESIGN DECISIONS

The system is divided into four independent components following the Single Responsibility Principle (SRP) and Dependency Inversion Principle (DIP). Inter-component communication is mediated through abstract interfaces; concrete classes are never directly coupled to one another.

Table I. Processing Pipeline Stages

Stage	Component	Responsibility
1	PEParser	Static PE header analysis
2	DebuggerEngine	Process launch, OEP detection
3	Dumper	Read sections from memory & save
4	IATRebuilder	Import table reconstruction
5	Dumper.SaveAs	Write dump file to disk

A. Applied Design Patterns

Strategy Pattern: The *IUnpacker* abstract interface allows different OEP-detection strategies (debugger, emulator, sandbox, etc.) to be swapped into the same pipeline. The current implementation instantiates this interface with *DebuggerEngine*.

Dependency Inversion Principle (DIP): The *Logger* class abstracts its output target via a *Sink* function object. The console application assigns `std::cout` as the sink; the GUI application assigns the Win32 message queue. Engine code therefore has zero dependency on the UI layer.

RAII: All Win32 HANDLE resources (process, thread, file handles) are released automatically through destructors, minimizing the risk of handle leaks.

Thread-Safe Singleton Variant: *Logger::Mutex()* uses an *inline static* storage class to maintain a single mutex instance for the program lifetime, preventing concurrent write races between the debug loop and UI threads.

III. COMPONENT TECHNICAL DETAILS

A. IUnpacker Interface

Defined in *IUnpacker.h*, this abstract interface exposes three pure virtual methods: **Run()** (blocking main loop), **IsOEPFound()**, and **GetOEP()**. Future unpack strategies can be substituted at runtime

through this interface without modifying the orchestrator.

B. DebuggerEngine

This class sits at the core of the Windows Debugging API. It spawns the target process with the `CREATE_SUSPENDED` flag, attaches as a debugger via `DebugActiveProcess()`, and drives the `WaitForDebugEvent / ContinueDebugEvent` loop.

1) Hardware Breakpoints (DR0-DR3)

DR7 is encoded per Intel SDM Vol.3 §17.2.4. For each slot, the local enable (L bit), condition (R/W bits), and length (LEN bits) are set independently. An Execute-type hardware breakpoint raises `EXCEPTION_SINGLE_STEP`; the DR6 status register identifies which slot fired.

2) PAGE_GUARD Mechanism

`VirtualProtectEx()` appends the `PAGE_GUARD` flag to the target memory region. The packer's first access to this region raises `STATUS_GUARD_PAGE_VIOLATION`; Windows automatically clears the flag. The faulting address is recorded as the OEP candidate and all threads are suspended via `SuspendThread()`.

Table II. DebuggerEngine Exception Handler Flow

Exception Code	Action
EXCEPTION_BREAKPOINT	First INT3 (loader) is silently consumed; subsequent ones are software BPs
EXCEPTION_SINGLE_STEP	DR6 is read; if Execute BP, OEP is recorded
STATUS_GUARD_PAGE_VIOLATION	Fault address is the OEP candidate; all threads suspended
EXCEPTION_ACCESS_VIOLATION	Forwarded to debugee unhandled
Others	DBG_EXCEPTION_NOT_HANDLED is returned

C. Dumper

The core challenge of memory dumping is the mismatch between disk and in-memory alignment: on disk sections are aligned to `FileAlignment` (typically 0x200), while in memory they are aligned to `SectionAlignment` (typically 0x1000). Dumper resolves this with a memory-aligned dump strategy:

- `FileAlignment` is set equal to `SectionAlignment`.
- `PointerToRawData[i] ← VirtualAddress[i]`
- `SizeOfRawData[i] ← align(VirtualSize[i], FileAlignment)`

This approach mirrors Scylla's default behavior and is fully valid within the PE specification. Both 32-bit and 64-bit PE formats are supported; architecture is auto-detected from the `OptionalHeader` magic value.

D. IATRebuilder

Packed executables frequently lack a valid Import Address Table (IAT) in the reconstructed dump. IATRebuilder dynamically rebuilds the table using a Scylla-style algorithm:

Steps 1-2: Loaded modules are enumerated via `EnumProcessModulesEx()`; each module's export table is read through `ReadProcessMemory()` to build an `address → (module, function)` map. `LoadLibrary()` is

intentionally avoided — the target and tool processes occupy separate address spaces.

Steps 3-4: The specified IAT region is walked pointer by pointer. Each entry that resolves to a valid export address is classified into a 'chunk' (a group of consecutive imports belonging to one DLL).

Steps 5-6: An `IMAGE_IMPORT_DESCRIPTOR` array, INT (Import Name Table), `IMAGE_IMPORT_BY_NAME` blocks, and DLL name strings are assembled into a single binary blob; injected into the Dumper as a new '.scy0' section; and `DataDirectory[IMPORT]` is updated accordingly.

IV. GUI IMPLEMENTATION

The tool is also available with a graphical user interface built on the native Win32 API. The `unpacker_gui` build target uses a `WinMain` entry point and presents the window directly without opening a console.

A. Architecture: WndProc Dispatcher

Win32 windows store the address of the `MainWindow` object in the `GWLP_USERDATA` field. The static `WndProcStatic()` dispatcher retrieves this address and forwards calls to the member method `WndProc()`, yielding an object-oriented Win32 wrapper.

B. Thread Safety

The unpack pipeline runs on a dedicated worker thread so the UI thread never freezes. Engine log messages are captured via `Logger::SetSink()` and forwarded to the UI thread through `PostMessage(WM_APP_LOG)`, satisfying Win32 thread-safety requirements.

C. Controls and User Experience

The interface includes target / output EXE pickers, an HW Execute BP address field, IAT rebuild VA / size fields, PE Analysis / START UNPACK buttons, and a multi-line read-only log area. When the input file picker is used, an output path is automatically suggested.

V. BUILD AND USAGE

A. Requirements

- Windows 10/11 (64-bit)
- MSVC 2022 (C++20 support, /W4 /permissive-)
- CMake >= 3.20
- Linked libraries: psapi, comctl32, comdlg32, shell32

B. CMake Configuration

The project defines two targets: **unpacker** (CLI) and **unpacker_gui** (Win32 GUI). Common engine source files (*PEParser.cpp*, *ProcessManager.cpp*, *DebuggerEngine.cpp*, *Dumper.cpp*, *IATRebuilder.cpp*) are grouped in the `ENGINE_SOURCES` variable and shared by both targets.

C. CLI Usage Example

```
unpacker.exe sample.exe unpacked.exe ^
--bp-exec 0x7FF71B9619C0 ^
```

```
--iat 0x7FF71B970000 0x400
```

Numeric values are accepted as decimal or 0x-prefixed hexadecimal. The `--bp-exec` option sets a hardware Execute breakpoint; `--iat` enables IAT reconstruction.

D. Exit Codes

Table III. Exit Code Meanings

Code	Meaning
0	Success
2	OEP not detected
3	OEP is below ImageBase
10	PE parsing error
11	Process management error
12	Memory dump error
13	IAT reconstruction error
99	Unexpected exception

VI. CONCLUSION

This work designed and implemented an object-oriented tool for unpacking packed Windows PE executables at runtime, written in C++20. The system is built around a modular architecture, SOLID design principles, and the RAII idiom.

DebuggerEngine combines hardware breakpoints and the `PAGE_GUARD` mechanism to achieve reliable OEP detection. Dumper transparently handles the disk-vs-memory alignment discrepancy, while IATRebuilder reconstructs the import table from runtime information using a Scylla-inspired algorithm. The GUI component operates without UI freezes thanks to its worker thread and `PostMessage`-based log delivery.

Potential future improvements include: separate source files for `PEParser` and `ProcessManager`, an emulator-based `IUnpacker` strategy,

code-coverage-based OEP validation, and multi-layer packer support.

VII. ACKNOWLEDGMENT

This project was completed as part of the Object-Oriented Programming course. I would like to thank my course instructor for the guidance and course materials provided throughout the semester.

VIII. REFERENCES

- [1] Microsoft Corporation, "Debug Event Handling," Windows Dev Docs, <https://learn.microsoft.com/en-us/windows/win32/debug/debug-events>, 2024.
- [2] Intel Corporation, "Intel(R) 64 and IA-32 Architectures Software Developer's Manual," Vol. 3, Chap. 17, 2024.
- [3] NtCore, "Scylla Import Reconstructor," <https://github.com/NtQuery/Scylla>, 2024.
- [4] ISO/IEC, "Programming Language -- C++," ISO/IEC 14882:2020 (C++20), 2020.
- [5] B. Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley, 2013.
- [6] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.
- [7] Microsoft Corporation, "Portable Executable (PE) Format Specification," <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>, 2024.